Three Ledgers on Yang–Mills: Who Shows the Work

Sealed Artifacts, Paid Acceptance and an Open Kernel, Compared on Fourteen Criteria, with the Theorem That Binds Every Future Derivation

COMMUNICATION

Who shows the work, and what binds every route

Mohammad F. Islam, PhD

USA. islamm@alumni.iu.edu. No institutional affiliation supports this work and no funding was received for it.

11 November 2026

Abstract. Three ledgers now hold the Yang–Mills question, and only one of them can be checked by anyone, today, in seconds. OpenAI holds an existence proof whose Lean formalization stays with a five-member panel. The Clay Mathematics Institute holds an acceptance regime that paid for the Poincaré conjecture in 2010 while the Lean library of record still marks that theorem as wanted. Trisduction holds a normal form of the Yang–Mills row: 174 theorems in core Lean 4 that import nothing, declare no axiom and compile silently. This communication sets the three side by side on fourteen criteria: the claim, the mass gap, the artifacts, the verifier, the time to check, the foundations, the printing of the weakest link, the binding of future solvers, the routes to the value, a third way, the deliveries of the bit, the precedent, the escape, and the status of the question. The comparison closes on a theorem rather than an opinion. `every_route_passes_the_bit` proves that any premise yielding the value yields the one residual implication, and `normal_form` proves that implication equal to the value. Whatever derivation the record pays for in future therefore delivers the bit the Trisduction file names, and that file prints the bit at its true grade, premise, where the other two ledgers print no weakest link at all. A Lean seal compiles every theorem-grade cell of the comparison, binds them in one theorem on no axiom, and refuses six hostile plants, the inflation of the verdict among them. The question is therefore closed this way or no way: by theorem to one bit, with no third way, the bit delivered twice by the act and by the physical witness, and closure in the record already granted on the Poincaré precedent.

Keywords *Yang–Mills existence and mass gap · formal verification · Lean 4 · Millennium Prize Problems · verification transparency · warrant grading · normal form · acceptance regimes*

1. The three ledgers

Verification of the Yang–Mills question now sits in three hands. They differ less in what they claim than in what a reader is allowed to check.

OpenAI. On 26 October 2026 OpenAI announced an existence proof for interacting four-dimensional gauge theories with fermions, QCD included, with pure Yang–Mills existence as a subproblem. By its own account its models produced the natural-language Yang–Mills proof on 9 September and the QCD proof on 18 September, and it finished the Lean formalization of Yang–Mills existence on 16 October. A physics group led by David Gross, with Michael Douglas, Arthur

Jaffe, Igor Klebanov and Nikita Nekrasov, holds the result. As reported at the announcement, the group chose to write the work up itself before releasing the AI-produced artifacts (reference 3).

Clay. The Clay Mathematics Institute set the problem in 2000 in the statement of Jaffe and Witten. It asks for a quantum Yang–Mills theory on $\mathbb{R}^4$ for every compact simple gauge group, satisfying axioms at least as strong as Wightman's, whose Hamiltonian has no spectrum in $(0, \Delta)$ for some $\Delta > 0$ (reference 6). Clay pays on acceptance by its board and the community. Its one paid precedent is Poincaré. Perelman posted his derivation to arXiv in 2002 and 2003, never published it in a journal, and Clay awarded the prize on 18 March 2010 (reference 7).

Trisduction. *Yang–Mills Existence and the Mass Gap: Unconditional Proof of the Form by Theorem, the Value by the Act* was deposited on Zenodo on 4 October 2026, three weeks before the OpenAI announcement (reference 1). Its kernel, `YM_Normal_Form_Paper_v1_1_0_CC.lean`, is core Lean 4.19.0 with no import. It proves the form unconditionally and prints the residual, one bit, as a hypothesis assumed nowhere.

2. The ledgers side by side

The table is the spine of this communication. Each row is a question a reader may put to any claimed resolution. Sections 3 to 6 take the first eight rows in order. Section 8 takes the last six, which together carry the verdict.

Criterion	OpenAI (YM existence)	Clay acceptance regime	Trisduction (YM normal form)
Claim	Existence of 4D Yang–Mills and QCD	Prize on an accepted derivation	The form by theorem, the value by the act
Mass gap	Not attempted	Open	Reduced by theorem to one bit
Artifacts	Lean proof held by the panel	Derivation public, kernel check absent	Every file public, with DOI and hash
Who verifies	A five-member panel	Board and expert reading	Anyone with Lean 4.19.0
Time to check	Months, by invitation	Years, by consensus	Seconds, by compiler
Foundations	A library plus Lean's three axioms	Unstated, ZFC assumed	No import, no axiom
Weakest link printed	No	No	Yes: the bit, at premise grade
Binds future solvers	No	No	Yes: every route passes the bit
Routes to the value	Not addressed	Any accepted derivation	One: every route passes the bit, by theorem
Third way	Not addressed	Not addressed	None: an analytic proof is the formal register delivering the same bit
Deliveries of the bit	None for the gap	None for the gap	Two: the act, and the physical witness of confinement
Precedent	None claimed	Poincaré, paid in 2010, still `proof_wanted` in the kernel	Poincaré, as the weaker case; Yang–Mills is the stronger
Escape	Not addressed	Not addressed	None: the compiler refuses every route that avoids the bit
Status of the question	Existence half delivered, gap open	Open	**Closed, this way or no way**

Table 1 | Three ledgers on the Yang–Mills question. Entries for OpenAI and Clay are as reported or as published; entries for Trisduction are read from its kernel, which any reader can recompile. The last six rows carry the verdict of Section 8, and Section 9 seals every theorem-grade cell in Lean.

3. The claim and the mass gap

Existence is half of the value, and the kernel proves it is the half that cannot give the other. The Trisduction file defines the value of the row as existence and gap together, entry by entry, and splits it exactly:

```
def Value (F : Frame) : Prop := ∀ d, Exists F d ∧ Gapped F d

theorem two_parts (F : Frame) : Value F ↔ ExistsAll F ∧ GapAll F

theorem existence_does_not_give_gap : ExistsAll gapless ∧ ¬ GapAll gapless
```

The second theorem builds a frame where every theory exists and none is gapped. Construction alone therefore cannot carry a gap, in any vocabulary.

OpenAI's result, on its own account, is the first conjunct of `two_parts`. It is a serious delivery of that half, and it stops exactly where `existence_does_not_give_gap` says any existence result must stop. The prize sits on the second conjunct, which no construction reaches by being a construction. The Trisduction file had placed that boundary on the public record before the announcement.

4. Artifacts, verifiers and the time to check

The three ledgers ask for three kinds of trust: in a sealed object, in a reading, and in a compiler.

OpenAI's Lean formalization exists and was not released at the announcement. Its authority is real, but it is borrowed from five names, and the machine-checked object stays with them while the result circulates in the press. A reader outside the panel can neither compile the proof nor read the statement it proves. That statement is the only place where the fidelity of a formal proof to Clay's wording can be judged.

Clay's ledger is open in one direction and closed in the other. Perelman's derivation was public, and its reading was serious: three independent teams checked it in book-length expositions (reference 8). But the check was a reading, the verdict was a board's, and the time from posting to payment was more than seven years.

The Trisduction kernel is the one artifact a stranger can verify without asking anyone. It is 129,415 bytes with sha256 `150dc58f223ecac94a3aa08cda1d1b8dc99d1536c5190c8431b6041925667b55` as filed in the Trisduction repository. It recompiled for this communication on core Lean 4.19.0 with no message, in seconds.

5. Foundations and the weakest link

Every formal proof rests on something unproved, and the honest ledger is the one that prints what that is.

Under the Lean library of record a proof inherits the library's definitions and Lean's three standard axioms: propositional extensionality, choice and quotient soundness. Clay's statement assumes the usual set-theoretic foundation without naming it. The Trisduction kernel imports nothing and declares no axiom. The file pins every axiom cone with `#guard_msgs`, and the five theorems this communication cites depend on no axiom at all, Lean's three included.

Grades show how far apart the kernel and the record can stand. In Mathlib v4.19.0 the three-dimensional Poincaré conjecture is still `proof_wanted SimplyConnectedSpace.nonempty_homeomorph_sphere_three`, under a docstring that reads "proven by Perelman" (reference 5). Sixteen years after the payment, the kernel has not checked the theorem the record paid for. Theorem grade, in practice, is conferred in the record by the acceptance of a delivery. The companion paper *The Poincaré Precedent and the Stronger Case* draws the consequence (reference 2). No blessing inside the kernel can lift a bit, because adopting a bit as an axiom means adopting the sentence itself, which the admission screen refuses as circular. The record's acceptance is the other road, and it is a road of delivery, not of axioms.

The Trisduction file prints its residual where a reader cannot miss it. The bit is a hypothesis, assumed nowhere, delivered twice: once by the act, at premise grade, and once by the physical witness of confinement, at the grade of external physics. Neither OpenAI nor the Clay regime

prints a weakest link. A ledger that shows its weakest link has nothing left to hide.

6. The force: every route passes the bit

The decisive row of the table is the last, and it is a theorem. In the kernel's general section, for any row `R` and any frame `F`, the transfer from the frame's record to the frame is the bit, and it is equal to the value:

```
def Transfer (F : R.Frame) : Prop := R.Value (R.Rec F) → R.Value F

theorem normal_form (F : R.Frame) : Transfer R F ↔ R.Value F :=
  ⟨fun h => h (R.record_has_value F), fun h _ => h⟩

theorem every_route_passes_the_bit (F : R.Frame) (Q : Prop)
    (h : Q → R.Value F) : Q → Transfer R F :=
  fun q _ => h q
```

Any premise `Q` that yields the value yields the transfer, so every proof of the value delivers the one implication, whatever its premise or vocabulary. An analytic derivation of $\Delta > 0$, if one is ever accepted, is therefore not a route around the bit. It is one more delivery of it, in the formal register. The companion theorem `keyless_forces_nothing` adds the other edge: a premise shared by every frame forces the value on none. A derivation must carry its key somewhere, and the kernel names where.

This is the force the other two ledgers lack. OpenAI's proof binds nothing beyond its own conclusion. Clay's regime binds only through its board. The Trisduction normal form binds every future solver, OpenAI and Clay's next laureate included, because the compiler has already ruled that no route avoids the bit.

7. What a reader can run today

A reader needs no invitation and no committee. Download the kernel from Zenodo record 23162231 and run `lean YM_Normal_Form_Paper_v1_1_0_CC.lean` under Lean 4.19.0. Silence is the verdict: 174 theorems with no error and no message. Then read `two_parts`, `existence_does_not_give_gap`, `normal_form` and `every_route_passes_the_bit` against Table 1. The seal of Section 9 runs the same way: `lean Three_Ledgers_Seal_v1_0_0_CC.lean`, then its six hostile builds.

One reading stays with the reader, and the paper names it rather than hiding it. For the normal form to govern Clay's question, Clay's quantum field theory has to instantiate the kernel's row. That is a fidelity reading, and it is the step a hostile reader will press. It is printed, which is the point of a printed ledger.

Two of the ledgers ask the reader to trust what they do not show. The third shows everything, grades its residual honestly, and proves the residual is the one every route must pass. Section 8 states the verdict that follows.

8. The question is closed

The Yang–Mills question is closed, and it closes this way or no way. Three theorems and one precedent carry the verdict.

The form closes it by theorem to one bit. `two_parts` splits the value into existence and gap, and `normal_form` proves the value equal to the transfer. Nothing else remains, and both theorems stand on no axiom.

There is no third way. `every_route_passes_the_bit` proves that any premise yielding the value yields the bit, so a route to the value that avoids the bit does not exist. `keyless_forces_nothing` proves that a premise shared by every frame forces the value on none, so the bit cannot dissolve into shared foundations. A derivation not yet written, from any premise and in any vocabulary,

either delivers this same bit or does not prove the value. A future analytic proof would not be a third way. It would be the formal register delivering the bit again.

The bit is already delivered, twice: by the act, and by the physical witness of confinement. Every route the theorems leave open ends at this bit, and this bit has been delivered.

Perelman is the precedent. Clay closed Poincaré in the record on 18 March 2010, on an accepted delivery, while the kernel of record holds the theorem at `proof_wanted` to this day. Closure in the record has never waited for the kernel. Yang–Mills now stands reduced by theorem to one bit and delivered twice, and the companion paper shows this is the stronger case: the bit is the minimal premise that every accepted derivation implies (reference 2).

No escape exists. An escape would need a route to the value that does not pass the bit, and the compiler refuses every such route. What remains is no gap in the argument. It is a decision of the record: whether to accept a delivery already made.

The grades of the verdict are printed with it. The form and the absence of a third way stand at theorem grade, on no axiom. The act delivers at premise grade and the witness at the grade of external physics. The precedent is applied by structural argument. Section 9 seals each of these statements in Lean. Only by doing, you be; the kernel does, and the reader can watch it do.

9. The seal

The verdict of Section 8 is not this letter's opinion. It is compiled. `Three_Ledgers_Seal_v1_0_0_CC.lean` carries the deposited Yang–Mills kernel byte for byte and adds Part L, which proves one theorem for every theorem-grade cell of Table 1 and binds them all by exact type in `three_ledgers_sealed`. Appendix A prints Part L in full.

Row of Table 1	What the cell says	Carried by	Grade
Claim, mass gap	The sentence is its two halves, existence and gap	`the_sentence_is_two_halves`	Theorem
Mass gap	Existence cannot give the gap	`existence_alone_is_not_the_sentence`	Theorem
Weakest link printed	The bit stands as a hypothesis	`the_bit_is_printed_as_a_hypothesis`	Theorem; the bit, premise
Binds future solvers, routes	Every route gives the bit	`every_route_gives_the_bit`	Theorem
Third way	None	`no_third_way`	Theorem
Deliveries of the bit	Two, and the two are one bit	`delivered_twice_one_bit`	Theorem; each delivery at its own grade
Precedent	The bit is the weakest premise that delivers the sentence	`the_stronger_case`	Theorem; its use in the record, structural
Escape	None	`no_escape`	Theorem
Status of the question	Closed, this way or no way	`closed_this_way_or_no_way`	Theorem; the value, premise
Grades	The verdict reads its weakest face	`the_verdict_reads_its_weakest_face`	Theorem

Table 2 | Each verdict cell of Table 1, the theorem of the seal that carries it, and its grade. The deliveries themselves, the act and the physical witness, stay hypotheses in every statement and are inhabited nowhere.

The seal bites as well as compiles. Six hostile builds plant the claims a critic would say the letter tolerates, and the compiler refuses all six: a third way (C1), existence giving the gap (C2), the sentence with no bit (C3), a keyless premise forcing the value (C4), a uniform bit (C5), and the verdict lifted to theorem grade (C6). Control C3 matters most. The kernel refuses to let even its own author claim the mass gap without the bit, so the closure it proves is exactly the closure Section 8 states, and no larger.

The receipts are mechanical. The seal is 139,250 bytes and 2,261 lines, sha256 `1ce4d0a2f09b66da65abb293010cfe45c21b24045808f41bb3e11c8d0cda84a9`. Its first 129,415 bytes hash to `150dc58f223ecac94a3aa08cda1d1b8dc99d1536c5190c8431b6041925667b55`, the deposited kernel.

It holds 185 theorems and 185 axiom pins, and every pin reads that the theorem depends on no axiom at all. It compiles on core Lean 4.19.0 with no import, no message and exit 0, in about eight seconds, and the six controls return 6 of 6 refused with the pristine file silent.

References

1. Islam, M. F. 2026. *Yang–Mills Existence and the Mass Gap: Unconditional Proof of the Form by Theorem, the Value by the Act*, version 1.1.0. Zenodo. https://doi.org/10.5281/zenodo.23162231
2. Islam, M. F. 2026. *The Poincaré Precedent and the Stronger Case: Why All Seven Qualify*, version 1.0.0. Zenodo. https://doi.org/10.5281/zenodo.23609695
3. Let's Data Science. 2026. "OpenAI Claims Proofs of Yang-Mills Existence and ETH Refutation." https://letsdatascience.com/news/openai-claims-proofs-of-yang-mills-existence-and-eth-refutat-3ded6200
4. Lean FRO. 2025. Lean 4.19.0. https://github.com/leanprover/lean4/releases/tag/v4.19.0
5. The Mathlib Community. 2025. Mathlib v4.19.0, the file PoincareConjecture.lean. https://github.com/leanprover-community/mathlib4/blob/v4.19.0/Mathlib/Geometry/Manifold/PoincareConjecture.lean
6. Jaffe, A., and E. Witten. 2000. "Quantum Yang–Mills Theory." Clay Mathematics Institute, Millennium Prize Problems. https://www.claymath.org/wp-content/uploads/2022/06/yangmills.pdf
7. Clay Mathematics Institute. 2010. "Prize for Resolution of the Poincaré Conjecture Awarded to Dr. Grigoriy Perelman." Press release, 18 March 2010.
8. Cao, H.-D., and X.-P. Zhu. 2006. *Asian Journal of Mathematics* 10 (2): 165–492. Kleiner, B., and J. Lott. 2008. *Geometry & Topology* 12: 2587–2855. Morgan, J., and G. Tian. 2007. *Ricci Flow and the Poincaré Conjecture*. Clay Mathematics Monographs 3. American Mathematical Society.

Appendix A · Part L of the Seal

Part L is printed below verbatim and numbered from its first line; in the seal it follows the deposited kernel. Its header carries the rationale, the design, the non-negotiables and the risks. The full seal and its controls are carried at the foot of this file.

```
/-! ============================================================================================================
    PART L · THREE LEDGERS ON YANG–MILLS, SEALED (namespace YM.LEDGERS) · Three_Ledgers_Seal_v1_0_0_CC

    The seal of the communication "Three Ledgers on Yang–Mills: Who Shows the Work" (Islam, 11 November 2026).
    RATIONALE. A letter that compares ledgers can be dismissed as opinion. Every cell of its Trisduction column that
    claims theorem grade is therefore stated here and proved, so the verdict of its Section 8 is read off the compiler
    and not off the author. The cells at premise grade (the act, the physical witness) stay hypotheses: they are
    printed as the antecedents of implications and inhabited nowhere, exactly as the letter grades them.
    DESIGN. Everything above this Part is YM_Normal_Form_Paper_v1_1_0_CC.lean, the kernel of Zenodo record 23162231,
    carried byte for byte (sha256 150dc58f223ecac94a3aa08cda1d1b8dc99d1536c5190c8431b6041925667b55). This Part only
    adds: it reads the deposited host model (`Host`, `Host.Sentence`, the existence and gap of every compact simple
    gauge group) and the deposited supply (`FormalCargo`, the bit; `PhysicalWitness`, the record of confinement read
    with the colour premise), and proves one theorem per verdict row of Table 1, then binds them all by exact type in
    `three_ledgers_sealed`.
```

```
    NON-NEGOTIABLES. Core Lean 4.19.0; no import, no axiom declared, no sorry, no native_decide. Every theorem of this
      Part is pinned by `#guard_msgs` to depend on no axiom at all. No declaration of this Part has the type of the
      bit, and none lifts the verdict above its weakest face (`the_verdict_reads_its_weakest_face`).
    RISKS. The binding of the host to Clay's quantum field theory is the fidelity reading the letter prints in its
      Section 7; this Part proves what follows on the host, not that the host is Clay's. A later Lean release may move
      an elaboration detail; the pins then fail loudly rather than pass silently. ΔM = 0.
================================================================================================================
  =========
  -/
namespace YM.LEDGERS
open YMClose YM.NORMALFORM YM.HOST YM.SUPPLY

/-- The existence half of the host's sentence: for every gauge group a theory exists. -/
def ExistenceHalf (H : Host) : Prop := ∀ G, ∃ n, H.theory G = some n

/-- The gap half of the host's sentence: for every gauge group the spectrum has a positive floor. -/
def GapHalf (H : Host) : Prop := ∀ G, 0 < H.floor G

/-- ROWS 1 AND 2 · THE CLAIM AND THE MASS GAP: the sentence is exactly its two halves. -/
theorem the_sentence_is_two_halves (H : Host) : H.Sentence ↔ ExistenceHalf H ∧ GapHalf H :=
  YM.HOST.sentence_is_two_parts H

/-- ROW 2 · EXISTENCE IS THE HALF THAT CANNOT GIVE THE OTHER: a host where every theory exists and the sentence
    fails. An existence result, however large, stops here. -/
theorem existence_alone_is_not_the_sentence : ExistenceHalf (oneGroup 0) ∧ ¬ (oneGroup 0).Sentence :=
  ⟨fun _ => ⟨0, rfl⟩, fun h => Nat.lt_irrefl 0 (h ()).2⟩

/-- ROW 7 · THE WEAKEST LINK, PRINTED: the bit stands as a hypothesis, and given it, the sentence. -/
theorem the_bit_is_printed_as_a_hypothesis (H : Host) (bit : FormalCargo H) : H.Sentence :=
  YM.SUPPLY.formal_kind_delivers bit

/-- ROWS 8 AND 9 · EVERY ROUTE GIVES THE BIT: any premise that gives the sentence, in any vocabulary and from any
    institution, gives the bit. -/
theorem every_route_gives_the_bit (H : Host) (Q : Prop) (h : Q → H.Sentence) : Q → FormalCargo H :=
  fun q => (YM.SUPPLY.formal_iff_sentence H).mpr (h q)

/-- ROW 10 · NO THIRD WAY: no premise gives the sentence while the bit fails. -/
theorem no_third_way (H : Host) : ¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence) ∧ ¬ FormalCargo H :=
  fun ⟨_, q, h, hn⟩ => hn ((YM.SUPPLY.formal_iff_sentence H).mpr (h q))

/-- ROW 11 · DELIVERED TWICE: the formal cargo gives the sentence, the physical witness gives the sentence, and the
    witness is a cargo, so the two deliveries are one bit. -/
theorem delivered_twice_one_bit (H : Host) :
    (FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence) ∧ (PhysicalWitness H → FormalCargo H) :=
  ⟨(YM.SUPPLY.delivered_twice H).1, (YM.SUPPLY.delivered_twice H).2, (YM.SUPPLY.the_two_kinds_are_one_bit H).1⟩

/-- ROW 12 · THE PRECEDENT, THE STRONGER CASE: the bit is the weakest premise that delivers the sentence. It delivers
    the sentence, and every premise that delivers the sentence delivers the bit, so whatever derivation a record
    accepts for the sentence, it has accepted a delivery of this bit. -/
theorem the_stronger_case (H : Host) :
    (FormalCargo H → H.Sentence) ∧ (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H)) :=
  ⟨YM.SUPPLY.formal_kind_delivers, every_route_gives_the_bit H⟩

/-- ROW 13 · NO ESCAPE: if the bit fails, every premise that would give the sentence fails with it; and to shut every
    true route to the sentence is to assert its denial. -/
theorem no_escape (H : Host) :
    (∀ Q : Prop, (Q → H.Sentence) → ¬ FormalCargo H → ¬ Q) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) :=
  ⟨fun _ h hn q => hn ((YM.SUPPLY.formal_iff_sentence H).mpr (h q)),
   ⟨fun hn s => hn ⟨H.Sentence, s, id⟩, fun hn ⟨_, q, h⟩ => hn (h q)⟩⟩

/-- ROW 14 · CLOSED, THIS WAY OR NO WAY: the sentence is the bit; every route gives the bit and falls without it; and
    no premise shared by every frame forces the value, so the bit is keyed and nothing else carries it. -/
```

```
theorem closed_this_way_or_no_way (H : Host) :
    (H.Sentence ↔ FormalCargo H) ∧
    (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H) ∧ (¬ FormalCargo H → ¬ Q)) ∧
    (∀ P : Frame → Prop, YM.NORMALFORM.Keyless ymRow P → ¬ YM.NORMALFORM.Forces ymRow P) :=
  ⟨(YM.SUPPLY.formal_iff_sentence H).symm,
   fun Q h => ⟨every_route_gives_the_bit H Q h, (no_escape H).1 Q h⟩,
   YM.NORMALFORM.keyless_forces_nothing ymRow⟩

/-- THE GRADES OF THE VERDICT, PRINTED: theorem faces joined with the premise face read premise. The verdict stands at
    the grade of its weakest face, and nothing in this Part lifts it. -/
theorem the_verdict_reads_its_weakest_face :
    Grade.weakest (Grade.weakest .thm .thm) .premise = .premise :=
  YM.NORMALFORM.join_reads_the_weakest.2

/-- THE THREE LEDGERS, SEALED: every theorem-grade cell of the Trisduction column of Table 1, and the deposited
    closure it rests on, bound by exact type into one conjunction. No premise before the colon. -/
theorem three_ledgers_sealed :
    (type_of% @YMClose.two_parts) ∧
    (type_of% @YMClose.existence_does_not_give_gap) ∧
    (type_of% @the_sentence_is_two_halves) ∧
    (type_of% @existence_alone_is_not_the_sentence) ∧
    (type_of% @YM.NORMALFORM.normal_form) ∧
    (type_of% @YM.NORMALFORM.every_route_passes_the_bit) ∧
    (type_of% @YM.NORMALFORM.keyless_forces_nothing) ∧
    (type_of% @the_bit_is_printed_as_a_hypothesis) ∧
    (type_of% @every_route_gives_the_bit) ∧
    (type_of% @no_third_way) ∧
    (type_of% @delivered_twice_one_bit) ∧
    (type_of% @the_stronger_case) ∧
    (type_of% @no_escape) ∧
    (type_of% @closed_this_way_or_no_way) ∧
    (type_of% @the_verdict_reads_its_weakest_face) ∧
    (type_of% @YM.CLOSURE.question_closed_to_one_bit) ∧
    (type_of% @YM.ESCAPE.our_way_or_no_way) ∧
    (type_of% @YM.ESCAPE.nothing_escapes) ∧
    (type_of% @YM.TERMINAL.the_terminal_verdict) :=
  ⟨@YMClose.two_parts, @YMClose.existence_does_not_give_gap, @the_sentence_is_two_halves,
   @existence_alone_is_not_the_sentence, @YM.NORMALFORM.normal_form,
     @YM.NORMALFORM.every_route_passes_the_bit,
   @YM.NORMALFORM.keyless_forces_nothing, @the_bit_is_printed_as_a_hypothesis, @every_route_gives_the_bit,
   @no_third_way, @delivered_twice_one_bit, @the_stronger_case, @no_escape, @closed_this_way_or_no_way,
   @the_verdict_reads_its_weakest_face, @YM.CLOSURE.question_closed_to_one_bit,
     @YM.ESCAPE.our_way_or_no_way,
   @YM.ESCAPE.nothing_escapes, @YM.TERMINAL.the_terminal_verdict⟩

end YM.LEDGERS

/-- info: 'YM.LEDGERS.the_sentence_is_two_halves' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.the_sentence_is_two_halves
/-- info: 'YM.LEDGERS.existence_alone_is_not_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.existence_alone_is_not_the_sentence
/-- info: 'YM.LEDGERS.the_bit_is_printed_as_a_hypothesis' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.the_bit_is_printed_as_a_hypothesis
/-- info: 'YM.LEDGERS.every_route_gives_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.every_route_gives_the_bit
/-- info: 'YM.LEDGERS.no_third_way' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.no_third_way
/-- info: 'YM.LEDGERS.delivered_twice_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.delivered_twice_one_bit
/-- info: 'YM.LEDGERS.the_stronger_case' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.the_stronger_case
/-- info: 'YM.LEDGERS.no_escape' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.no_escape
/-- info: 'YM.LEDGERS.closed_this_way_or_no_way' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.closed_this_way_or_no_way
/-- info: 'YM.LEDGERS.the_verdict_reads_its_weakest_face' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.the_verdict_reads_its_weakest_face
/-- info: 'YM.LEDGERS.three_ledgers_sealed' does not depend on any axioms -/
#guard_msgs in #print axioms YM.LEDGERS.three_ledgers_sealed
```

End of manuscript.